

The Efficacy of NLinear Neural Network Over the Performer, a Transformer Variant: A Study in Bitcoin Price Prediction

Student: Huseyn Gorbani
SID NO. : 220913779
Supervisor: Dr Fredrik Dahlqvist
MSc Artificial Intelligence
Queen Mary University of London
School of Electronic Engineering and
Computer Science

Abstract—Transformers have become widely used architecture in natural language processing tasks, which has resulted in the proposal of a wide variety of architecture variations in the field of time series prediction. These variants claim to produce cutting-edge outcomes. This study assesses the efficacy of the Performer, a recent state-of-the-art version of the Transformer, against vanilla Transformers and neural networks. To establish this comparison, a diverse dataset is prepared for training and evaluation purposes. It is found that Normalisation-Linear (NLinear) outperforms the Performer model in nearly all cases. This work emphasises how simpler models could potentially be more suitable than more complex and sophisticated ones, especially when it comes to real-time prediction. This study not only provides direction for future research in Bitcoin price prediction, but it encourages to utilise the proposed dataset. By doing this, it is possible to create a common standard for comparison, fostering solutions that the broader community can employ to improve prediction performance.

Keywords—Bitcoin, transformers, Performer, neural networks, LTSF-Linear, financial forecasting, cryptocurrency, real-time prediction

I. INTRODUCTION

The field of time series forecasting has witnessed significant strides, with cryptocurrency price prediction being a central focus in recent years. Bitcoin (ticker name: BTC), the first and leading cryptocurrency, has sparked myriad research opportunities since its inception in 2009 by an anonymous entity known as Satoshi Nakamoto (Nakamoto, 2008). Consequently, this has led to the application of diverse methodologies for price prediction.

Similar to the majority of cryptocurrencies, the volatile nature of Bitcoin presents distinct challenges during trading. Its attractiveness has grown as a result of the numerous people who have made large profits with Bitcoin. But predicting its value with any degree of accuracy is still a difficult task. The literature on forecasting Bitcoin prices does not, as far as the author of this work is aware, contain a standard dataset. The ability to consistently assess improvements in research in this field is made more difficult by this gap. The introduction of an extensive Bitcoin dataset is one of the main contributions of this study. The prepared dataset includes information on Bitcoin prices as well as relevant variables, sentiment data from news sources, commodities, market indices, and well-known indicators such as RSI and MACD. The work first focuses on the data collection and pre-processing procedures. The provided diverse dataset consists of 48 different features, making it suitable for deep learning models.

Additionally, this study compares the Performer, a novel Transformer variant designed by Du, Su and Wei (2023) specifically for time series forecasting, with custom-created simple Neural Networks, called LTSF-Linear models, developed by Zeng et al. (2023). The goal is to evaluate whether the Performer offers concrete benefits so that its complex architecture with long training and tuning process can be compromised. Both univariate and multivariate data inputs are used in these evaluations, highlighting any noticeable differences in prediction accuracy. Given Performer's sophisticated design compared to conventional neural network models, a key question in this study is whether in the case of achieving the highest accuracy with Performer, justifies its increased complexity, particularly in real-time trading circumstances. Last but not least, this research aims to determine whether just relying only on Bitcoin's closing price acts as a reliable feature for its future values.

In Section 2 of the paper, recent developments in cryptocurrency prediction and Transformer variants are explored very briefly. The methodology is described in Section 3 in detail, including information on data collection and pre-processing, model architectures, and training and evaluation procedures. In the results section, the vanilla Transformer is assessed, various models are compared, and empirical testing on Bitcoin price prediction is reviewed. A conclusion and recommendations for further research are offered in Sections 5 and 6, respectively.

II. RELATED WORK

The prediction of Bitcoin prices has been a topic of interest in recent years, with various researchers employing different methodologies to tackle this problem. Gupta and Nain (2021) focused on predicting Bitcoin prices using a variety of machine learning and time series analysis techniques, including Simple Average, Moving Average, Weighted Average, Naive Approach, Linear Regression, Polynomial Regression, Support Vector Regression (SVR), Long Short-Term Memory (LSTM), and Gated Recurrent Unit (GRU). They found that the Naive Approach performed the best, with an accuracy of 86.78%, followed closely by the Weighted Average method with an accuracy of 86.32%. Interestingly, the LSTM model had an accuracy of 33.84%.

Another study by Shin, Mohaisen and Kim (2021) proposed an ensemble-enabled LSTM model for Bitcoin price prediction, which considers various time scales. This approach differs from previous works in that it considers various time scales for price prediction. Authors of this work believed that “most of the previous works have used

information such as price, volume, and text contents for price forecasting, and focused on data sets that have moderate shifts.” The authors evaluated the performance of the proposed model over two distinct three-minute intervals. According to their findings, the ensemble-enabled LSTM demonstrated an average performance rate of 96.21% in terms of 1-MAPE (a variant of Mean Absolute Percentage Error used for accuracy assessments) and achieved a score of 34.42 in RSME (Root Mean Square Error).

Mudassir et al. (2020) focused on short-term to mid-term Bitcoin price forecasts using machine learning models. They achieved fairly accurate end-of-day, short-term (7 days), and mid-term (30 and 90 days) Bitcoin price forecasts using machine learning models, including Artificial Neural Networks (ANN), Stacked Artificial Neural Networks (SANN), SVM, and LSTM. This work provides a comprehensive view of various machine learning models' performance in predicting Bitcoin prices. It is concluded that among machine learning models, LSTM was particularly more effective in forecasting Bitcoin price by achieving MAPE as low as 1.44%. However, accurately predicting the rise and fall of the Bitcoin price trend still remained a challenge for the authors.

A different approach by Maleki et al. (2020) focused on predicting Bitcoin prices based on three other well-known coins (i.e., Ethereum, Zcash, and Litecoin) using machine learning algorithms and time series analysis. They found that among the three cryptocurrencies, Zcash had the most similar trend to Bitcoin. This study's findings could provide a new perspective on Bitcoin price prediction by considering the trends of other cryptocurrencies.

Another study by Dixon and London (2021) introduced a variant of the standard RNN, called α -RNNs, or Exponentially Smoothed Recurrent Neural Networks. The α -RNN is a variant of the standard RNN that introduces a smoothing parameter, α , which is used to control the contribution of the current input and the past hidden state to the new hidden state. This mechanism helps α -RNNs to better capture long-term dependencies in the data compared to standard RNNs. It is concluded that a simple α -RNN can simulate non-stationary time series better than GRUs or LSTMs in high-frequency trading. In addition, the suggested model requires little hyperparameter optimisation and provides outstanding outcomes without the need for data pre-processing.

The advent of deep learning techniques in time series forecasting has shown considerable promise in various studies. Lim and Zohren (2021) explored different encoder architectures like CNNs, RNNs, and attention-based models. Their study highlighted the potential of attention mechanisms, particularly the Transformer model, in boosting the performance of time-series forecasting. A comprehensive survey, by Wen et al. (2022) offered a taxonomy to summarize the various modifications and application domains of existing time series Transformers. Despite these advances, the efficacy of Transformer-based models in long-term forecasting was questioned by Zeng et al. (2023), who proposed a simpler model, LTSF-Linear, for comparison. They argue that most existing Transformer based models fail to extract temporal relations/dependencies from long sequences, meaning the forecasting error are not reduced but rather increased when increasing the size of the look-back window.

Diving deeper into Bitcoin price prediction, Tanwar and Kumar (2022) combined LSTM and Transformers to forecast financial time series for Bitcoin and other cryptocurrencies. Their approach was claimed to be more accurate than other existing methods. However, this method, despite its accuracy, required longer processing times, being unsuitable for real-time forecasting tasks.

Addressing computational limitations, Liu et al. (2021) introduced the Pyraformer model, a variant of the Transformer model. The Pyraformer model is especially designed to mitigate time and memory complexity issues when training on time series dataset and it is proved to be particularly effective for both single-step and long-range multi-step time series forecasting tasks. The use of a pyramidal graph structure allowed the model to consider dependencies of different ranges in a multi-resolution manner, providing more accurate predictions on various real-world datasets compared to other Transformer variants. A notable application of the Pyraformer model was demonstrated by Dong et al. (2023). They harnessed its predictive power to forecast the risk level of heavy metal contamination in wheat in China, effectively bridging a gap in food safety risk prediction. Their study not only affirmed the model's high accuracy but also showcased its capacity to grade risk levels. While their experiment was not in a financial context, their innovative use of Pyraformer provides valuable insights for other predictive tasks, such as Bitcoin price forecasting, thus illustrating the adaptability of the Pyraformer model across diverse fields.

Another promising model was the Predictive Transformer (Preformer) introduced by Du, Su and Wei (2023) developed for long-term time series forecasting. This model introduced a novel predictive paradigm along with an efficient attention mechanism, Multi-Scale Segment-Correlation (MSSC), performing better than other models in nearly all cases, particularly in long prediction horizons.

While these models offer promising results, there's an ongoing debate regarding the efficiency and complexity of deep learning models in forecasting. In light of this, Sridhar and Sanagavarapu (2021) adopted a multi-head self-attention Transformer model in the domain of cryptocurrency price prediction. Their model, trained on hourly Dogecoin cryptocurrency data, yielded an impressive accuracy rate (1-MAPE) of 98.46% and an RMSE value of 25.873.

In the field of forecasting hybrid models are recognised for their potential. Hybrid models merge traditional time series and deep learning models (Lim and Zohren 2021). These models adjust their internal settings or 'model parameters' at every step using deep neural networks, learning from data over time. They also incorporate expert knowledge, using historical data or insights to guide their predictions. This not only enhances their learning but also narrows down the range of possible outcomes they consider, making their predictions more accurate. In essence, hybrid models represent a powerful blend of statistical methods, advanced learning systems, and expert insights.

While deep learning techniques and attention mechanisms present substantial potential in time series forecasting for cryptocurrency price prediction, there is a persistent debate over the trade-off between model complexity and performance. Assessing their effectiveness and potential limitations is necessary, especially in the realm of real-time forecasting. At the same time, the exploration and testing of

potentially efficient and less complex models, such as LTSF-Linear, could offer valuable insights into the dynamic field of time series forecasting.

Approaches such as the ensemble-enabled LSTM model and α -RNNs, which consider various time scales and incorporate smoothing parameters respectively, represent innovative methods in forecasting. Other strategies, like the one proposed by Maleki et al. (2020), highlight the possibility of leveraging the trends of other cryptocurrencies for Bitcoin price prediction.

Notably, the research conducted by Gupta and Nain (2021) underscores the importance of continuing the exploration of both deep learning models and simpler techniques, such as the Naive Approach or the Weighted Average method. These varying approaches contribute to our expanding comprehension of the intricate task of Bitcoin price prediction.

III. METHODOLOGY

A. Data Collection and Feature Selection

Data plays a pivotal role in the performance of deep learning models. For architectures such as Transformers, an ample amount of data points is essential for ensuring effectiveness. Granular Bitcoin OHCL (i.e., Open, High, Close, Low) price data was initially obtained from Bitstamp.net, one of the leading cryptocurrency exchanges. This data, which was originally in minute intervals, was converted to an hourly resolution, spanning from January 1st, 2012, to May 21st, 2023.

To establish a comprehensive and robust dataset, additional on-chain and external information related to Bitcoin and other prominent cryptocurrencies, such as Ethereum, was retrieved. This decision was inspired from the study done by Kim et al. (2022), which provided a selected set of variables from Glassnode.com. Glassnode is one of the major data sources that provides investors with insights into the blockchain and cryptocurrency sectors. By the time this project was undertaken, most of the variables identified by Kim et al. (2022) were accessible via Glassnode's API in both daily and hourly resolutions.

Beyond cryptocurrency data, information on various indices, commodities, and metrics, such as the USA federal funds rate and inflation rate, was gathered from various platforms including Yahoo Finance. Also, considering the significant influence of news on Bitcoin's price which is shaped by the decision of retail investors, extensive historical news sentiment data was collected from Snapi, a news sentiment data provider. This work utilised the services of Snapi to access historical sentiment data on cryptocurrencies¹, foreign exchange² (Forex) and stock market³.

After the data collection stage, the Glassnode data underwent an initial pre-processing phase. First, using the MinMaxScaler function from Python's sklearn library, each feature in the dataset was normalised to a scale between 0 and 1. This ensured that all features were on the same numerical scale. Following normalisation, Granger's causality test with a maximum lag of 7 was employed to determine the potential predictive power of every feature collected from Glassnode in relation to the close price of Bitcoin. However, prior to conducting the causality test, the stationarity of each time series data (i.e., each feature) was verified using the Augmented Dickey-Fuller (ADF) test, setting p-value ≤ 0.05 . A series was considered stationary if its p-value was 0.05 or

below, implying rejection of the null hypothesis of the ADF test at the 5% level of significance. To achieve stationarity, any non-stationary series underwent one differencing, ensuring the validity of Granger's causality test. Subsequent evaluations of Granger's causality test results focused on p-values ≥ 0.05 . Features with such p-values were assumed to have little Granger-causal influence on Bitcoin's price and were thus eliminated. Through this pre-processing, the dataset from Glassnode was carefully refined to retain only features that might have a predictive relationship with Bitcoin's price. Please refer to Appendix A for a comprehensive list of non-stationary series from Glassnode.com, series excluded due to causality test p-values being 0.05 or higher, and features exhibiting Granger's causality on Bitcoin's closing price.

When dealing with commodities and indices datasets, NaN values presented a significant challenge, especially when they sporadically appear among other non-missing values. Due to this scattered occurrence, where some data points are NaN while others are not, we cannot simply eliminate entire rows. Removing rows haphazardly may damage the integrity of the data and have a negative impact on the performance and accuracy of deep learning models. Hence, a sophisticated approach to address these missing values is essential. To navigate this challenge, the K-Nearest Neighbours (KNN) imputation method was adopted. The mean of the 10 closest neighbours is used to impute missing values. The nan euclidean metric is used to measure "nearness". The method doesn't necessarily consider temporal order, and the 10 neighbours aren't necessarily five on either side but rather the ones who are closest to one another in feature space. All neighbours contribute equally to the value that is imputed by default, which is a uniform weighting. This robust pre-processing guarantees that the dataset is properly prepared for future analysis, with its handled missing values.

In the analysis of Bitcoin's closing prices, some common technical indicators were also incorporated as potential features to better understand its market dynamics. A 24-hour Simple Moving Average (SMA) was calculated, offering insights into the daily price trend by smoothing out short-term volatilities. In addition, a 24-hour Relative Strength Index (RSI) was computed, providing a measure of the daily price momentum and potentially highlighting overbought or oversold scenarios. Using its default 12-period and 26-period Exponential Moving Averages (EMAs), the Moving Average Convergence Divergence (MACD) displayed changes in Bitcoin's momentum, which is potentially helpful for locating possible buying or selling positions. A 168-hour (or 7-day) SMA and RSI were added to widen the scope and provide a more comprehensive picture of Bitcoin's weekly direction and momentum. Following the incorporation of these technical indicators, any data gaps, represented by NaN values, were filled in using a back-fill approach to guarantee a continuous and complete dataset prepared for in-depth studies. Appendix A provides a detailed list of all the features after the end of the pre-processing step of data for this study.

Moreover, given the considerable number of data points (99,793) in the hourly pre-processed data, the dataset was divided into training, validation, and testing sets in ratios of 90%, 5%, and 5%, respectively. To refine the dataset further, a hyperparameter-tuned encoder-only vanilla Transformers model was utilised. This model accepted the Bitcoin close price along with one feature at a time to determine which feature combination performs the best. Following this

Links to Snapi LLC services:

1. <https://cryptonews-api.com>
2. <https://forexnewsapi.com>
3. <https://stocknewsapi.com>

experiment, the top 10 features and Bitcoin close price were selected to train deep learning models.

B. Models' Architecture

It's crucial to establish a baseline before adopting the most recent deep learning models for Bitcoin price prediction. In order to do this, the encoder-only vanilla Transformer model was chosen as a starting point. This will be used as a benchmark against the vanilla Transformer (encoder-decoder version), LTSF-Linear models, and Preformer.

To start from the baseline model, it made use of the encoder component of standard Transformer architecture. The first layer in the design is an embedding layer, which is implemented as a linear layer that converts input features into a vector representation the Transformer can use. The introduction of a positional encoding stage is necessary since self-attention does not inherently understand sequence order. Positional encoding adds positional information to embeddings in Transformers. To ensure that model considers sequence order, sine and cosine functions are used for even ($2i$) and odd ($2i + 1$) indices, respectively:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

where d_{model} is the embedding dimension and pos represents the position in the embedding sequence.

Once the input has been processed with both embedding and positional encoding, it's directed to the encoder part of the model. Following encoding, the mean of each encoded sequence and the last value in the sequence are merged, forming an aggregation similar to a residual connection observed in many other systems. The multilayer perceptron (MLP) is then fed with this aggregated result. The MLP is composed of a linear layer with ReLU activation, a dropout layer for regularisation, and a final linear layer to produce the desired output. Overall, the base model is an integration of the self-attention mechanism from Transformers and a subsequent MLP.

The vanilla Transformer's encoder-decoder version is then put into practice. Similar to the baseline, this model incorporates a positional encoding layer and an embedding layer to preserve sequence order. Separately created embedding layers for the encoder and decoder are used to process inputs. Following the encoding process, the decoder uses the "teacher forcing" technique. The likelihood that the decoder will input its own prior predictions or the actual output during training is equal. This approach ensures robustness by letting the model benefit from ground truth sequences while also learning from its mistakes. The mean and the last value from the prediction sequence are merged after the decoding stage, and the resulting value is then fed through a multilayer perceptron (MLP) to produce the final prediction.

In this work, two variants of LTSF-Linear are also trained which are introduced by Zeng et al. (2023): 1) Decomposition-Linear (DLinear), 2) Normalisation-Linear (NLinear). DLinear and NLinear models present distinct methodologies for handling different data characteristics. For datasets with observable trends, the DLinear model is designed. This is accomplished by splitting the data into two components: a trend segment and a seasonal segment. It can be modelled mathematically as follows:

$$x(t) = trend(t) + seasonal(t)$$

the time series value at time t is represented by $x(t)$, and the trend and seasonality component at time t is represented by $trend(t)$ and $seasonal(t)$, respectively. Each of these components is then processed through distinct linear models as follows:

$$y_{trend}(t) = W_{trend} \cdot trend(t) + b,$$

$$y_{seasonal}(t) = W_{seasonal} \cdot seasonal(t) + b'$$

where W_{trend} and $W_{seasonal}$ are the weight matrices and b and b' are the bias terms for the trend and seasonality components respectively. Following this, the final prediction is obtained by adding these two processed parts:

$$y(t) = y_{trend}(t) + y_{seasonal}(t).$$

where $y(t)$ is the predicted sequence, of which its length is set as a hyperparameter. In this work, DLinear employs dedicated linear layers for each component across various channels (i.e., features in the dataset), underlining its precision-oriented approach.

The NLinear model, on the other hand, is strong at handling datasets that exhibit distributional shifts. To ensure that the input to the linear layer remains consistent despite potential data variations, it first normalises the sequence in relation to its last value. This is denoted mathematically as:

$$x_{normalised}(t) = x(t) - x_{last}(t)$$

where $x_{normalised}(t)$ is the normalised value at time t , and $x_{last}(t)$ is the last value in the sequence. Next, a linear layer is used to process this normalised input:

$$y_{normalised}(t) = W_{normalised} \cdot x_{normalised}(t) + d$$

where d is the bias term and $W_{normalised}$ is the weight matrix. After this, the sequence is rescaled by reintroducing the original distributional characteristics:

$$y(t) = y_{normalised}(t) + x_{last}(t).$$

where $y(t)$ is the predicted sequence, of which its length is set as a hyperparameter. To optimise the handling of various data aspects, NLinear assigns specialised linear transformations to each individual channel. In short, NLinear is made to manage data that may vary unexpectedly over time, whereas DLinear is built for data with clear trends.

Finally, the Preformer, a state-of-the-art Transformer model variant designed for time series forecasting was trained on the prepared Bitcoin dataset. The Preformer model offers a novel approach called Multi-Scale Segment-Correlation (MSSC), which split time series into segments and employs segment-level correlation-based attention instead of point-wise attention. This method is introduced on the idea that since time series data exhibit notable internal dynamics and continuity at a local level, within a segment, stronger correlations can be seen at segment level. By switching from point-wise attention to segment-wise attention, MSSC determines the cross-correlation between two segments as a metric of similarity, hence using less computational power.

To explore the model in mathematical terms based on the original work done by Du, Su and Wei (2023), MSSC takes the input of each Segment-Correlation (SC) module, denoted as $H \in \mathbb{R}^{L \times d}$, where L and d are the length and dimension of the input, respectively. SC module is essentially an attention

mechanism that aggregates information based on the segments of input data while maintaining the integrity of/continuity within each segment.

For the purpose of computing attention scores in Transformer-based models, the query, key, and value are linearly transformed representations of the input data. The query, key, and value are obtained by projecting the input series H using three projection matrices (i.e., $Q = HW_q$, $K = HW_k$, $V = HW_v$). These projections are divided into several segments of the same length:

$$L_{seg} : \{Q_1, Q_2, \dots, Q_m\}, \{K_1, K_2, \dots, K_n\}, \{V_1, V_2, \dots, V_n\}.$$

where $Q_i \in \mathbb{R}^{L_{seg} \times d}$, $K_i \in \mathbb{R}^{L_{seg} \times d}$, $V_i \in \mathbb{R}^{L_{seg} \times d}$, m, n indicate the number of segments, and as a hyperparameter L_{seg} representing length of a segment, determines the computational complexity in the Preformer model. (Du, Su and Wei 2023) Furthermore, even though it is not explicitly mentioned by the authors, how W_q , W_k , W_v are initialised, after analysing the source code it is evident that they follow a standard approach which is using Xavier Uniform initialisation provided in PyTorch by default.

The following formula is provided to calculate the correlation measurement between any two query (Q_i) and key (K_j) segments:

$$c_{ij} = \text{Correlation}(Q_i, K_j), \\ \text{Correlation}(Q_i, K_j) = \frac{1}{d \times L_{seg}} Q_i \cdot K_j.$$

The Softmax function normalises the correlation measures of each query segment Q_i with all key segments to provide the aggregation weight:

$$\hat{c}_{i1}, \hat{c}_{i2}, \dots, \hat{c}_{in} = \text{Softmax}(c_{i1}, c_{i2}, \dots, c_{in}).$$

Following that, the output at the i -th segment, Y_i , is the weighted sum of all value segments:

$$Y_i = \sum_{j=1}^n \hat{c}_{ij} V_j$$

Next, all the Y_i along the length dimension are concatenated to produce the output of the segment-correlation module:

$$SC(H; L_{seg}) = \text{Concatenate}(Y_1, Y_2, \dots, Y_m)$$

By combining the output of several Segment-Correlations with various L_{seg} , MSSC expands Segment-Correlation. Given that the length of a segment determines the smallest unit engaged in Segment-Correlation, a larger L_{seg} captures coarse-grained temporal relationships, whilst a smaller L_{seg} captures fine-grained dependencies. When the scale level rises, an initial segment length L_0 is selected and the segment length rises exponentially, which is denoted by: $L_l = 2^l L_0$ where l , the scale level, falls between the range of 0 to l_{max} . The outputs for the various scales are produced by processing the input H through these Segment-Correlation layers of varying scale levels. The MSSC equation is:

$$MSSC(H) = \sum_{l=0}^{l_{max}} \frac{2^l}{\sum_{l=0}^{l_{max}} 2^l} \cdot SC(H; 2^l L_0)$$

By adding a segment delay between the keys and their corresponding values during the decoding phase, Preformer

introduces a Predictive Paradigm. To acquire the result of the current segment Y_i , the query of the preceding segment Q_{i-1} is utilised to calculate correlations with the keys of all segments. This suggests that the value segment actually comes from the segment that follows the current key segment that is used in the calculation. In light of this offset relationship between keys and values, the output for the current segment Y_i is calculated as follows:

$$Y_i = \sum_{j=1}^{n-1} \hat{c}_{(Q_{i-1}, K_j)} V_{j+1}$$

In essence, Preformer uses MSSC and Predictive Multi-Scale Segment-Correlation (PreMSSC) instead of the vanilla Transformer model's original self-attention and cross-attention. This makes efficient forecasting possible by allowing an unknown prediction segment to build the query on its own and predict itself while utilising correlations between other segments.

It is important to highlight that only vanilla Transformers are implemented from scratch using PyTorch. This is because many studies in the literature do not seem to accurately represent vanilla Transformers. Self-attention mechanisms and neural networks should be the sole components of vanilla Transformers, with no convolutional or pooling layers. Additionally, since the source code of other models (i.e., LTSF-Linear¹ and Preformer² models) has already been provided by the authors, it was unnecessary to implement them again. Therefore, the Preformer model source code has been merged into the LTSF-Linear repository which is available as one of the sub-directories in this project's source code. This is merely for the sake of avoiding repetition and giving priority to models' comparison.

C. Training and Evaluation

In this study, hyperparameter tuning was applied to both vanilla Transformers and performer models. For the LTSF linear models, hyperparameter tuning was not undertaken as they have very limited hyperparameters in comparison to other models. Instead, the models' hyperparameters were chosen manually. For the rest of the models though, the Tree of Parzen Estimators (TPE) algorithm, available in Python's Hyperopt library, was employed. The advantage of this algorithm over alternatives such as random or grid search is that it can intelligently sample hyperparameters based on previous assessments, often resulting in a quicker and more efficient convergence to optimal values. The hyperparameter tuning was conducted using the top 10 features and the Bitcoin close price features, with 100 trials for each model. The vanilla Transformers also have early stopping, which kicks in when the loss function doesn't seem to be getting any better. The Adam and Mean Square Error (MSE) are used as the optimiser function and loss function for vanilla Transformers, respectively. MSE is also employed as the metric for hyperparameter adjustment. Both Mean Square Error (MSE) and Mean Absolute Error (MAE) were incorporated for all models' evaluation and comparison.

D. Specification of Development Environment

An Intel(R) Xeon(R) CPU E5-2620 v3 @ 2.40GHz with 12 physical cores and 24 threads, as well as 62 GiB of RAM, were used as the computational environment during the development. A single NVIDIA GeForce RTX 2070 was used out of 4 available GPUs. Rocky Linux 8.8 (Green Obsidian)

Original source code of LTSF-Linear and Preformer:

1. <https://github.com/cure-lab/LTSF-Linear>
2. <https://github.com/ddz16/Preformer/tree/main>

with Kernel version 4.18.0-477.10.1.el8_8.x86_64 and CUDA version 12.1 were utilised for this experiment.

IV. RESULTS

A. Importance of Embedding in Time Series Forecasting

Before starting the first experiment, it is important to understand that the data in this work is entirely numerical, unlike tasks such as natural language processing where words lack inherent meaning without token embeddings. This study first investigated whether embedding would improve vanilla Transformer’s accuracy performance in time series forecasting. To this end, two encoder-only Transformers were trained—one with an embedding layer and the other without—on the top five columns, and Bitcoin close price. Refer to Fig. 1 and Fig. 2 in Appendix B for a visual representation of predictions made using these two versions of the encoder-only Transformer on the testing dataset. The results showed that the forecasts without the embedding layer were quite inaccurate and seasonal. However, after incorporating the embedding layer, the predictions greatly improved in precision and matched the actual ground truth more closely.

B. Evaluation of Vanilla Transformer on Varied Bitcoin Feature Combinations

In the first experiment, vanilla Transformers were trained and tested. These Transformers were hyperparameter-tuned, and trained on a variety of feature sets, each integrated with the Bitcoin close column: alone, with the top two features, with the top five features, and with the top ten features. For detailed hyperparameter results, refer to Appendix C, which covers it both for the vanilla Transformers and the Performer model. Then, both the encoder-only and encoder-decoder variants of the Transformer were assessed on these feature sets (see Table II). In each iteration of this experiment, an input sequence of 96 was used to predict the subsequent Bitcoin close price. The primary aim was to determine whether the close price feature by itself was sufficient or if a combination with other variables offered deeper insights for the deep learning models. The findings of this experiment are shown in the following table.

TABLE I. COMPARISON OF VANILLA TRANSFORMERS ON VARIOUS FEATURE COMBINATIONS

Feature Combinations (Input/Output: 96/1)	Encoder Only Vanilla Transformer		Encoder-Decoder Vanilla Transformer	
	MSE	MAE	MSE	MAE
Only Bitcoin Close Price	700792.81	601.87	1129581.87	872.17
Top 2 Features & BTC Close	1965491.87	1167.16	1410454.75	970.03
Top 5 Features & BTC Close	1156210.87	838.08	923090.18	744.97
Top 10 Features & BTC Close	5603149	1914.76	6063901	2260.82

In this experiment, the encoder-only vanilla Transformer performed the best when only the Bitcoin close price feature was provided. The encoder-decoder variant of the vanilla Transformer showed commendable results with just the Bitcoin close price, especially when compared to its performance with the top 10 and top 2 feature combinations. Yet, its peak performance was achieved on the top 5 features and the Bitcoin close price combined. One might conclude that using the Bitcoin close price alone may be adequate for forecasts given that the encoder-only Transformer had lower

MSE and MAE values (indicating higher performance) than its encoder-decoder equivalent. This outcome may be due to the fact that while extra features can provide information, they may also create noise and harm the model’s performance. Although the finding was convincing further research was necessary. As a result, the next experiment focused on training the vanilla Transformer, LTSF-Linear, and Performer models using only the close price, and its combinations with selected top 10 features.

C. Evaluation of All Models

In this study, expanding on the leanings from the previous experiment, models were trained and tested under the following input/output settings:

- An input length of 12 predicting the immediate subsequent step (I/O: 12/1).
- An input length of 12 predicting the next 12th step (I/O: 12/12).
- An input length of 96 predicting the immediate subsequent step (I/O: 96/1).
- An input length of 96 predicting the next 12th step (I/O: 96/12).
- An input length of 504 forecasting the next 12th step (I/O: 504/12).

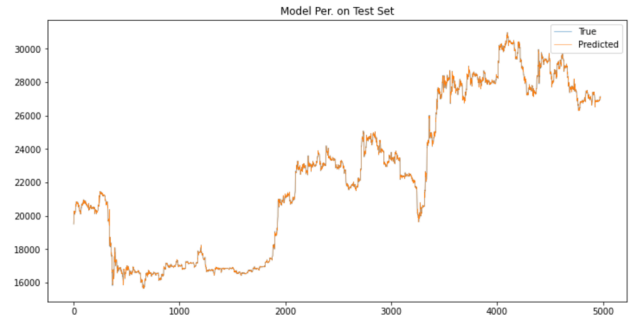


Fig 1. Performance of the NLinear model on predicting Bitcoin’s next closing price using a 12/1 I/O setting, trained exclusively on Bitcoin close prices.

Upon examining Table II and Table III, it is evident that the NLinear model, despite its simplicity and lightweight nature, exhibited remarkable performance in nearly all scenarios—both with the bitcoin close price (see Fig .1) and when top features were combined with the bitcoin close price. It is difficult to say whether the Performer model or the DLinear model produced the second-best outcomes. A thorough analysis of the MSE and MAE scores, however, reveals that the Performer performs better than DLinear. Notably, the Performer does not demonstrate overfitting to the same degree as the DLinear model when the input size or number of features is rising. To further clarify, as the input size or feature count rises, the MSE and MAE values for DLinear climb noticeably more than those for the Performer.

In the majority of instances, training the models on top features combined with the Bitcoin close price, negatively impacted their performance—with the notable exception of the DLinear model. Surprisingly, training the DLinear model on top features and the close price as compared to just the close price seems to improve performance. This finding is intriguing because it was predicted that DLinear would have

TABLE II. COMPARISON OF ALL MODELS ON VARIOUS INPUT AND OUTPUT SETTINGS WHEN ONLY TRAINED ON BITCOIN CLOSE PRICE

Input/Output (I/O) Settings – BTC Close Price Only	Encoder Only Vanilla Transformer		Encoder Decoder Vanilla Transformer		Decomposition-Linear (DLLinear)		Normalisation-Linear (NLinear)		Predictive Transformer (Preformer)	
	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
I/O: 12/1	492610.71	568.2	168751.90	323.31	11342.58	68.24	63.32	5.02	1916.37	34.44
I/O: 12/12	465627.25	532.34	1206645.75	952.62	83550.61	169.06	78518.70	156.02	62179.21	164.02
I/O: 96/1	1370571.25	958.65	958898.68	755.98	4252488	1442.86	11099.50	74.03	787082.81	583.61
I/O: 96/12	877252.06	679.34	1289945.75	898.42	3218322.25	1249.47	96567.50	192.49	414962.40	420.88
I/O: 504/12	OFM ^a	OFM ^a	4057064	1732.05	229810880	11505.36	1188066.87	742.79	OFM ^a	OFM ^a

^a Short for out of memory.

TABLE III. COMPARISON OF ALL MODELS ON VARIOUS INPUT AND OUTPUT SETTINGS WHEN TRAINED ON TOP FEATURES AND BITCOIN CLOSE PRICE

Input/Output (I/O) Settings – BTC Close Price & Top Features	Encoder Only Vanilla Transformer		Encoder Decoder Vanilla Transformer		Decomposition-Linear (DLLinear)		Normalisation-Linear (NLinear)		Predictive Transformer (Preformer)	
	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
I/O: 12/1	14410662	3661.32	4294490	1608.09	3500.11	38.25	85.77	5.91	21806.99	104.71
I/O: 12/12	7229045	2481.1	576629.12	570.20	79940	160.31	78371.87	155.73	77326.34	165.85
I/O: 96/1	2159156	4304.32	2024030.12	1132.92	3986668.5	1389.92	4850.99	49.47	609738.43	513.62
I/O: 96/12	4512943	1763.52	5150821.5	2036.71	3023244	1192.68	88156.42	177.91	492087	468.76
I/O: 504/12	OFM ^a	OFM ^a	OFM ^a	OFM ^a	217425840	11037.66	1822908.37	828.61	OFM ^a	OFM ^a

^a Short for out of memory.

an overfitting tendency, just like the other models, prior to the experiment. However, this assumption proved to be incorrect. The situation with the NLinear and Preformer models is different. The incorporation of multiple features seemingly brought a degree of overfitting or introduced noise into these models, with one exception that when the input sequence size was set at 96 with predictions targeting the next time step. In this particular input setting, both the NLinear and Preformer models witnessed a decrease in their MSE and MAE values, indicating improved performance.

Another interesting observation arises in the context of the 12/1 and 96/1 input/output settings. As shown in Table II, focusing only on the close price, both the MSE and MAE values increase when the input size grows, while the output size (i.e., that being the next step, size of one) remains constant. This observation suggests that providing an excessive amount of input to the model each time, just to forecast the next step could potentially lead to overfitting. Similarly, in experiments where the model is trained on top features along with the close price, a similar tendency is seen, except for the encoder-decoder version of the vanilla Transformer. This is also the case with the 12/12 and 96/12 settings, except for the encoder-only vanilla Transformer exhibiting improvement with a 96 input length when trained on top features along with the close price, refer to Table III.

Finally, when the input size was set to 504 and the output size to 12, all Transformer-based experiments encountered a "CUDA out of memory" error, except for the encoder-decoder version trained only on the Bitcoin close price feature. This indicates that there was not enough GPU RAM. The choice was made to restrict the experiment to the single GPU present on the shared server, even though using multiple GPUs may

have solved this problem. This highlighted a problem with Transformer-based models when handling large input sizes.

D. Empirical Testing

Because of how well the NLinear model performed, it was feasible to use it in a real-world situation. The Streamlit framework was used to create a simple web application that makes this possible. This single web page application employs a candlestick chart to display hourly Bitcoin OHCL prices in real-time, with the model's predictions shown in yellow. The 12/12 I/O NLinear (only trained on Bitcoin close price) model was used in this experiment to forecast the next 12-hour trajectory. The following extraction logic is used to select predictions: Given P as the set of all predictions, and $|P|$ as the length (number of elements) in P , the mathematical representation could be:

$$\mathcal{Y}_{preds} = \{P_i | i = 12n, \text{ where } n \in \mathbb{N}_0 \text{ and } i \neq P_{|P|} - 12\} \cup \{P_{|P|-11}, P_{|P|-10}, \dots, P_{|P|}\}$$

where $\mathbb{N}_0$ is the set of non-negative integers, the first set $\{P_i | i = 12n, \text{ where } n \in \mathbb{N}_0 \text{ and } i \neq P_{|P|} - 12\}$ represents every 12th element except last one in the sequence and the second set $\{P_{|P|-11}, P_{|P|-10}, \dots, P_{|P|}\}$ represents the last 12 elements. This effectively captures every 12th-hour forecast alongside the final 12 predictions. The main objective was to show an estimated 12-hour price movement for Bitcoin. While the model does an adequate job of making these predictions, it finds it difficult to capture the full volatility and unpredictability of Bitcoin's market trends. Determining the exact times when Bitcoin prices fluctuate is still a difficult task. For instance, in one case, a price trajectory that was predicted actually occurred after a week, crossing the threshold of \$30,000 (See Fig. 3 and Fig. 4 in Appendix D).

This case emphasises a key point: even one of the best performing models may not always be able to predict the precise timing or volatility of changes in the price of bitcoin, even though it may be indicative of general trends.

V. CONCLUSION

In this study, a diverse dataset was first prepared to evaluate how well different models predicted Bitcoin prices. Based on metrics like MSE and MAE, the results showed that most models performed better when only considering the close price feature, with the exception of the DLinear model. The MSE and MAE values in the DLinear model, however, decreased when additional top features were added. Another finding suggests that models' performance suffered when the input size was increased, for example, from 12 to 96. Overall, models' ability to precisely predict short-term trajectories is degraded, which may be attributed to the increased noise.

The sophisticated Preformer model, a cutting-edge Transformer variant made for time series forecasting, couldn't compete with the NLinear model despite its inherent simplicity, which outperformed all other models as well. Being more time-efficient as well as simpler to train and evaluate, LTSF-Linear models won out in terms of efficiency.

A key finding from empirical testing was that while models may perform well on test datasets, their accuracy in the real world can suffer, especially during uncertain market phases. This becomes even more evident when focusing primarily on the close price, which can only provide a small glimpse of the broader market dynamics. There were some limitations to the study's scope, most notably the lack of consultation with professionals in Bitcoin trading and market analysis. They may have been able to suggest more important market factors and improve the data collection with their insights. Additionally, conventional methodologies—often used in literature to evaluate model performance—might not always be compatible with the actual difficulties of forecasting real-time market movements.

In conclusion, despite NLinear's promising performance in predictive accuracy compared to Preformer, a thorough and multifaceted approach is necessary for accurate forecasting due to the inherent complexities and uncertainties of the Bitcoin market.

VI. FEATURE WORK

The study identifies potential directions for future investigation into Bitcoin price prediction. Although the Bitcoin price feature was found to be adequate for forecasting in this study, additional features may be useful to explore in future research to possibly improve prediction accuracy. In addition, moving away from hourly data points and investigating various granularity levels may reveal unexpected patterns and insights. Furthermore, integrating reinforcement learning mechanisms with existing prediction models could also be beneficial, providing a more robust prediction framework. The idea of pre-trained models, particularly an ensemble of NLinears tailored for specific tasks like Bitcoin price prediction, remains an exciting proposition. In future works, it might be possible to predict Bitcoin price more accurately if the guidance of expert Bitcoin traders is combined with the implementation of the NLinear models.

REFERENCES

- Dixon, M. & London, J. 2021, "Financial forecasting with α -RNNs: a time series modeling approach", *Frontiers in Applied Mathematics and Statistics*, vol. 6, pp. 551138.
- Dong, W., Hu, T., Zhang, Q., Deng, F., Wang, M., Kong, J. & Dai, Y. 2023, "Prediction of Food Safety Risk Level of Wheat in China Based on Pyraformer Neural Network Model for Heavy Metal Contamination", *Foods*, vol. 12, no. 9, pp. 1843.
- Du, D., Su, B. & Wei, Z. 2023, "Preformer: predictive transformer with multi-scale segment-wise correlations for long-term time series forecasting", *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)IEEE*, , pp. 1.
- Gupta, A. & Nain, H. 2021, "Bitcoin price prediction using time series analysis and machine learning techniques", *Machine Learning for Predictive Analysis: Proceedings of ICTIS 2020*Springer, , pp. 551.
- Kim, G., Shin, D., Choi, J.G. & Lim, S. 2022, "A deep learning-based cryptocurrency price prediction model that uses on-chain data", *IEEE Access*, vol. 10, pp. 56232-56248.
- Lim, B. & Zohren, S. 2021, "Time-series forecasting with deep learning: a survey", *Philosophical Transactions of the Royal Society A*, vol. 379, no. 2194, pp. 20200209.
- Liu, S., Yu, H., Liao, C., Li, J., Lin, W., Liu, A.X. & Dustdar, S. 2021, "Pyraformer: Low-complexity pyramidal attention for long-range time series modeling and forecasting", *International conference on learning representations*.
- Maleki, N., Nikoubin, A., Rabbani, M. & Zeinali, Y. 2020, "Bitcoin price prediction based on other cryptocurrencies using machine learning and time series analysis", *Scientia Iranica*, .
- Mudassir, M., Bennbaia, S., Unal, D. & Hammoudeh, M. 2020, "Time-series forecasting of Bitcoin prices using high-dimensional features: a machine learning approach", *Neural computing and applications*, , pp. 1-15.
- Nakamoto, S. 2008, "Bitcoin: A peer-to-peer electronic cash system", *Decentralized business review*, .
- Shin, M., Mohaisen, D. & Kim, J. 2021, "Bitcoin price forecasting via ensemble-based LSTM deep learning networks", *2021 International conference on information networking (ICOIN)IEEE*, , pp. 603.
- Sridhar, S. & Sanagavarapu, S. 2021, "Multi-head self-attention transformer for dogecoin price prediction", *2021 14th International Conference on Human System Interaction (HSI)IEEE*, , pp. 1.
- Tanwar, A. & Kumar, V. 2022, "Prediction of Cryptocurrency prices using Transformers and Long Short term Neural Networks", *2022 International Conference on Intelligent Controller and Computing for Smart Power (ICICCS)IEEE*, , pp. 1.
- Wen, Q., Zhou, T., Zhang, C., Chen, W., Ma, Z., Yan, J. & Sun, L. 2022, "Transformers in time series: A survey", *arXiv preprint arXiv:2202.07125*, .

Zeng, A., Chen, M., Zhang, L. & Xu, Q. 2023, "Are transformers effective for time series forecasting?", *Proceedings of the AAAI conference on artificial intelligence*, pp. 11121.

APPENDICES

Appendix A

TABLE I. LIST OF ALL NON-STATIONARY FEATURES FROM GLASSNODE.COM

Feature NO.	Name of Features	Description of Features
1	Hash Rate	The average estimated number of hashes per second produced by the miners in the Bitcoin network. ¹
2	Active Addresses	The number of unique Bitcoin addresses that were active in the network either as a sender or receiver. Only addresses that were active in successful transactions are counted. ¹
3	Supply Last Active 1+ Years Ago	The percent of circulating supply that has not moved in at least 1 year. ¹
4	Transaction Size (Total)	The total size of all Bitcoin transactions within the time period (in bytes). ¹
5	Transaction Count	The total amount of transactions. Only successful transactions are counted. ¹ (Bitcoin)
6	Transaction Rate	The total amount of transactions per second. Only successful transactions are counted. ¹ (Bitcoin)
7	Stock-to-Flow Deflection	The Stock to Flow (S/F) Deflection is the ratio between the current Bitcoin price and the S/F model . If deflection is ≥ 1 it means that Bitcoin is overvalued according to the S/F model, otherwise undervalued. ¹
8	Net Unrealized Profit/Loss (NUPL)	Net Unrealized Profit/Loss is the difference between Relative Unrealized Profit and Relative Unrealized Loss. ¹
9	Exchange Balance (Percent)	The percent supply held on exchange addresses. ¹
10	Exchange Withdrawals	The total count of Bitcoin transfers from exchange addresses, i.e. the number of on-chain withdrawals from exchanges. ¹
11	Total Addresses	The total number of unique addresses that ever appeared in a transaction of the Bitcoin coin in the network. ¹
12	Difficulty	The current estimated number of hashes required to mine a block. ¹
13	Total Addresses (USDT)	The total number of unique addresses that ever appeared in a transaction of the USDT coin in the network. ¹
14	Transfer Volume (USDT)	The total amount of USDT coins transferred on-chain. Only successful transfers are counted. ¹
15	Market Cap	The market capitalisation (or network value) is defined as the product of the current supply by the current USD price. ¹ (Bitcoin)
16	Delta Cap	Delta Cap is the difference between Realized Cap and Average Cap, where Average Cap is assumed to be the life-to-date moving average of Market Cap. It attempts to detect major market bottoms in Bitcoin cycles. ¹
17	Realized Cap	Realized Price is the Realized Cap divided by the current supply. ¹ (Bitcoin)
18	Investor Capitalization	Investor Capitalization is the difference of Realized Cap and Thermocap. It discounts the capital paid to miners from the market's general cost basis, serving as a bottom indicator in bear cycles. ¹
19	Balanced Price	Balanced Price is the difference between Realized Price and Transfer Price. Transfer Price is the cumulative sum of Coin Days Destroyed in USD, adjusted by circulating supply and total time since Bitcoin's inception. Balanced Price attempts to detect major cycle bottoms. ¹
20	Realized Price	Realized Price is the Realized Cap divided by the current supply. ¹ (Bitcoin)
21	Relative Unrealized Profit	Relative Unrealized Profit is defined as the total profit in USD of all coins in existence whose price at realisation time was lower than the current price normalized by the market cap. ¹ (Bitcoin)
22	Supply Last Active 3y-5y	The amount of circulating supply last moved between 3 years and 5 years ago. ¹ (Bitcoin)
23	Supply last active 5y-7y	The amount of circulating supply last moved between 5 years and 7 years ago. ¹ (Bitcoin)
24	Market Cap (USDT)	The market capitalisation (or network value) is defined as the product of the current supply by the current USD price. ¹ (USDT)
25	Exchange Inflow Volume (Total) (USDT)	The total amount of USDT transferred to exchange addresses. ¹
26	Exchange Outflow Volume (Total) (USDT)	The total amount of USDT transferred from exchange addresses. ¹
27	Circulating Supply	The total amount of all USDT coins ever created/issued, i.e., the circulating supply. ¹
28	Price (ETH)	Ethereum closing price in USD. ¹
29	Price (LTC)	Litecoin closing price in USD. ¹
30	Price	Bitcoin closing price in USD. ¹
31	HODL Waves 1y_2y	Percentage of Bitcoin that last moved between 1 to 2 years ago, as part of the HODL waves concept introduced by Unchained Capital. ¹
32	HODL Waves 2y_3y	Percentage of Bitcoin that last moved between 2 to 3 years ago, as part of the HODL waves concept introduced by Unchained Capital. ¹
33	HODL Waves 3y_5y	Percentage of Bitcoin that last moved between 3 to 5 years ago, as part of the HODL waves concept introduced by Unchained Capital. ¹
34	HODL Waves 5y_7y	Percentage of Bitcoin that last moved between 5 to 7 years ago, as part of the HODL waves concept introduced by Unchained Capital. ¹

Feature NO.	Name of Features	Description of Features
35	HODL Waves 7y_10y	Percentage of Bitcoin that last moved between 7 to 10 years ago, as part of the HODL waves concept introduced by Unchained Capital. ¹
36	HODL Waves more_10y	Percentage of Bitcoin that last moved 10 years ago or more, as part of the HODL waves concept introduced by Unchained Capital. ¹
37	Realized Cap HODL Waves 1y_2y	Percentage of Bitcoin, weighted by its Realized Price, that last transacted between 1 to 2 years ago. ¹
38	Realized Cap HODL Waves 2y_3y	Percentage of Bitcoin, weighted by its Realized Price, that last transacted between 2 to 3 years ago. ¹
39	Realized Cap HODL Waves 3y_5y	Percentage of Bitcoin, weighted by its Realized Price, that last transacted between 3 to 5 years ago. ¹
40	Realized Cap HODL Waves 5y_7y	Percentage of Bitcoin, weighted by its Realized Price, that last transacted between 5 to 7 years ago. ¹
41	Realized Cap HODL Waves 7y_10y	Percentage of Bitcoin, weighted by its Realized Price, that last transacted between 7 to 10 years ago. ¹
42	Stock-to-Flow Ratio price	Stock to Flow is defined as the ratio of the current stock of a commodity (i.e., circulating Bitcoin supply) and the flow of new production (i.e. newly mined bitcoins). Bitcoin's price has historically followed the S/F Ratio and therefore it is a model that can be used to predict future Bitcoin valuations. ¹ (USD)
43	Stock-to-Flow Ratio ratio	Stock to Flow is defined as the ratio of the current stock of a commodity (i.e., circulating Bitcoin supply) and the flow of new production (i.e. newly mined bitcoins). Bitcoin's price has historically followed the S/F Ratio and therefore it is a model that can be used to predict future Bitcoin valuations. ¹ (Ratio)
44	close	Last price at which Bitcoin is traded (daily resolution).
45	high	Highest price at which Bitcoin is traded (daily resolution).
46	low	Lowest price at which Bitcoin is traded (daily resolution).
47	open	First price at which Bitcoin is traded (daily resolution).

¹Glassnode Studio (2023) Metric Catalog [online]. Available at: <https://studio.glassnode.com/catalog> [Accessed: 13 August 2023].

TABLE II. LIST OF ALL FEATURES FROM GLASSNODE.COM OF WHICH THEIR CAUSALITY TEST P-VALUE IS EQUAL TO OR MORE THAN 0.05 (MAX LAG BEING 7)

Feature NO.	Name of Features	Description of Features
1	Balanced Price	Balanced Price is the difference between Realized Price and Transfer Price. Transfer Price is the cumulative sum of Coin Days Destroyed in USD, adjusted by circulating supply and total time since Bitcoin's inception. Balanced Price attempts to detect major cycle bottoms. ¹
2	Block Height	The block height, i.e. the total number of blocks ever created and included in the main blockchain. ¹ (Bitcoin)
3	Delta Cap	Delta Cap is the difference between Realized Cap and Average Cap, where Average Cap is assumed to be the life-to-date moving average of Market Cap. It attempts to detect major market bottoms in Bitcoin cycles. ¹
4	Exchange Balance (Percent)	The percent supply held on exchange addresses. ¹
5	Exchange Deposits	The total count of transfers to exchange addresses, i.e. the number of on-chain deposits to exchanges. ¹ (Bitcoin)
6	Exchange Inflow Volume (Total)	The total amount of Bitcoin transferred to exchange addresses. ¹
7	Exchange Net Position Change	The 30d change of the supply held in exchange wallets. ¹ (Bitcoin)
8	Exchange Outflow Volume (Total)	The total amount of Bitcoin transferred from exchange addresses. ¹
9	HODL Waves 1d_1w	Percentage of Bitcoin that last moved between 1 day to 1 week ago, as part of the HODL waves concept. ¹
10	HODL Waves 1m_3m	Percentage of Bitcoin that last moved between 1 month to 3 months ago, as part of the HODL waves concept. ¹
11	HODL Waves 1w_1m	Percentage of Bitcoin that last moved between 1 week to 1 month ago, as part of the HODL waves concept. ¹
12	HODL Waves 1y_2y	Percentage of Bitcoin that last moved between 1 year to 2 years ago, as part of the HODL waves concept. ¹
13	HODL Waves 24h	Percentage of Bitcoin that last moved within the past 24 hours, as part of the HODL waves concept. ¹
14	HODL Waves 2y_3y	Percentage of Bitcoin that last moved between 2 years to 3 years ago, as part of the HODL waves concept. ¹
15	HODL Waves 3m_6m	Percentage of Bitcoin that last moved between 3 months to 6 months ago, as part of the HODL waves concept. ¹
16	HODL Waves 3y_5y	Percentage of Bitcoin that last moved between 3 years to 5 years ago, as part of the HODL waves concept. ¹
17	HODL Waves 5y_7y	Percentage of Bitcoin that last moved between 5 years to 7 years ago, as part of the HODL waves concept. ¹
18	HODL Waves 6m_12m	Percentage of Bitcoin that last moved between 6 months to 12 months ago, as part of the HODL waves concept. ¹
19	HODL Waves 7y_10y	Percentage of Bitcoin that last moved between 7 years to 10 years ago, as part of the HODL waves concept. ¹

Feature NO.	Name of Features	Description of Features
20	HODL Waves more_10y	Percentage of Bitcoin that last moved 10 years ago or more, as part of the HODL waves concept. ¹
21	Investor Capitalization	Investor Capitalization is the difference of Realized Cap and Thermocap. It discounts the capital paid to miners from the market's general cost basis, serving as a bottom indicator in bear cycles. ¹
22	MVRV Ratio	Market Value to Realized Value (MVRV) is the ratio between market cap and realised cap. It gives an indication of when the traded price is below a "fair value". ¹ (Bitcoin)
23	MVRV Z-Score	The MVRV Z-Score evaluates whether Bitcoin is overvalued or undervalued relative to its "fair value". Instead of using a traditional z-score method, the MVRV Z-Score uniquely compares the market value to the realized value. When the market value, measured as network valuation by spot price multiplied by supply, is significantly higher than the realized value, represented by the cumulative capital inflow into the asset, it has typically signaled a market top (red zone). Conversely, a significantly lower market value than the realized value has often indicated market bottoms (green zone). The MVRV Z-Score is defined as the ratio between the difference of market cap and realized cap, and the standard deviation of market cap, i.e. $(\text{market cap} - \text{realized cap}) / \text{std}(\text{market cap})$. This formula calculates the standard deviation from the first available data point until the present day, making it a cumulative measure. ¹
24	Market Cap (USDT)	The market capitalisation (or network value) is defined as the product of the current supply by the current USD price. ¹ (USDT)
25	Net Unrealized Profit/Loss (NUPL)	Net Unrealized Profit/Loss is the difference between Relative Unrealized Profit and Relative Unrealized Loss. ¹
26	Price Drawdown from ATH	The percent drawdown of the asset's price from the previous all-time high. ¹ (Bitcoin)
27	Realized Cap	Realized Price is the Realized Cap divided by the current supply. ¹ (Bitcoin)
28	Realized Cap HODL Waves 1d_1w	Percentage of Bitcoin, weighted by its Realized Price, that last transacted between 1 day to 1 week ago. ¹
29	Realized Cap HODL Waves 1m_3m	Percentage of Bitcoin, weighted by its Realized Price, that last transacted between 1 month to 3 months ago. ¹
30	Realized Cap HODL Waves 1w_1m	Percentage of Bitcoin, weighted by its Realized Price, that last transacted between 1 week to 1 month ago. ¹
31	Realized Cap HODL Waves 1y_2y	Percentage of Bitcoin, weighted by its Realized Price, that last transacted between 1 to 2 years ago. ¹
32	Realized Cap HODL Waves 24h	Percentage of Bitcoin, weighted by its Realized Price, that transacted in last 24 hours. ¹
33	Realized Cap HODL Waves 2y_3y	Percentage of Bitcoin, weighted by its Realized Price, that last transacted between 2 to 3 years ago. ¹
34	Realized Cap HODL Waves 3m_6m	Percentage of Bitcoin, weighted by its Realized Price, that last transacted between 3 to 6 months ago. ¹
35	Realized Cap HODL Waves 3y_5y	Percentage of Bitcoin, weighted by its Realized Price, that last transacted between 3 to 5 years ago. ¹
36	Realized Cap HODL Waves 5y_7y	Percentage of Bitcoin, weighted by its Realized Price, that last transacted between 5 to 7 years ago. ¹
37	Realized Cap HODL Waves 6m_12m	Percentage of Bitcoin, weighted by its Realized Price, that last transacted between 6 to 12 months ago. ¹
38	Realized Cap HODL Waves more_10y	Percentage of Bitcoin, weighted by its Realized Price, that last transacted 10 years or more ago. ¹
39	Realized Price	Realized Price is the Realized Cap divided by the current supply. ¹ (Bitcoin)
40	Relative Unrealized Profit	Relative Unrealized Profit is defined as the total profit in USD of all coins in existence whose price at realisation time was lower than the current price normalized by the market cap. ¹ (Bitcoin)
41	SOPR	The Spent Output Profit Ratio (SOPR) is computed by dividing the realized value (in USD) divided by the value at creation (USD) of a spent output. Or simply: price sold / price paid. ¹ (Bitcoin)
42	Stock-to-Flow Deflection	The Stock to Flow (S/F) Deflection is the ratio between the current Bitcoin price and the S/F model . If deflection is ≥ 1 it means that Bitcoin is overvalued according to the S/F model, otherwise undervalued. ¹
43	Stock-to-Flow Ratio daysTillHalving	Number of days left until the next Bitcoin halving event, which affects the rate of new Bitcoin production. ¹
44	Supply Last Active 1+ Years Ago	The percent of circulating supply that has not moved in at least 1 year. ¹ (Bitcoin)
45	Supply Last Active 3+ Years Ago	The percent of circulating supply that has not moved in at least 3 years. ¹ (Bitcoin)
46	Supply Last Active 3y-5y	The amount of circulating supply last moved between 3 years and 5 years ago. ¹ (Bitcoin)
47	Supply in Smart Contracts	The percent of total supply of the token that is held in smart contracts. ¹ (USDT)
48	Supply last active 5y-7y	The amount of circulating supply last moved between 5 years and 7 years ago. ¹ (Bitcoin)
49	Total Addresses	The total number of unique addresses that ever appeared in a transaction of the Bitcoin coin in the network. ¹
50	Total Addresses (USDT)	The total number of unique addresses that ever appeared in a transaction of the USDT coin in the network. ¹
51	Transaction Count	The total amount of transactions. Only successful transactions are counted. ¹ (Bitcoin)
52	Transaction Rate	The total amount of transactions per second. Only successful transactions are counted. ¹ (Bitcoin)
53	aSOPR	Adjusted SOPR is SOPR ignoring all outputs with a lifespan of less than 1 hour. ¹ (Bitcoin)

Feature NO.	Name of Features	Description of Features
54	volume	Number of Bitcoin traded (daily resolution)

¹Glassnode Studio (2023) Metric Catalog [online]. Available at: <https://studio.glassnode.com/catalog> [Accessed: 13 August 2023].

TABLE III. LIST OF ALL FEATURES THAT GRANGER-CAUSE BITCOIN CLOSE PRICE

Feature NO.	Name of Features	Description of Features
1	Hash Rate	The average estimated number of hashes per second produced by the miners in the Bitcoin network. ¹
2	Active Addresses	The number of unique Bitcoin addresses that were active in the network either as a sender or receiver. Only addresses that were active in successful transactions are counted. ¹
3	Transaction Size (Total)	The total size of all Bitcoin transactions within the time period (in bytes). ¹
4	Exchange Withdrawals	The total count of Bitcoin transfers from exchange addresses, i.e. the number of on-chain withdrawals from exchanges. ¹
5	Difficulty	The current estimated number of hashes required to mine a block. ¹
6	Transfer Volume (USDT)	The total amount of USDT coins transferred on-chain. Only successful transfers are counted. ¹
7	Market Cap	The market capitalisation of Bitcoin as the product of the current supply by the current USD price. ¹
8	Exchange Inflow Volume (Total) (USDT)	The total amount of USDT transferred to exchange addresses. ¹
9	Exchange Outflow Volume (Total) (USDT)	The total amount of USDT transferred from exchange addresses. ¹
10	Circulating Supply	The total amount of all USDT coins ever created/issued, i.e. the circulating supply. ¹
11	Price (ETH)	Ethereum closing price in USD. ¹
12	Price (LTC)	Litecoin closing price in USD. ¹
13	Price	Bitcoin closing price in USD. ¹
14	Realized Cap HODL Waves 7y 10y	7 to 10 years HODL waves weighted by realized price. ¹
15	Stock-to-Flow Ratio price	Stock to Flow is defined as the ratio of the current stock of a commodity (i.e. circulating Bitcoin supply) and the flow of new production (i.e. newly mined bitcoins). Bitcoin's price has historically followed the S/F Ratio and therefore it is a model that can be used to predict future Bitcoin valuations. ¹ (USD)
16	Stock-to-Flow Ratio ratio	Stock to Flow is defined as the ratio of the current stock of a commodity (i.e. circulating Bitcoin supply) and the flow of new production (i.e. newly mined bitcoins). Bitcoin's price has historically followed the S/F Ratio and therefore it is a model that can be used to predict future Bitcoin valuations. ¹ (Ratio)
17	close	Last price at which Bitcoin is traded on daily resolution.
18	high	Highest price at which Bitcoin is traded on daily resolution.
19	low	Lowest price at which Bitcoin is traded on daily resolution.
20	open	First price at which Bitcoin is traded on daily resolution.

¹Glassnode Studio (2023) Metric Catalog [online]. Available at: <https://studio.glassnode.com/catalog> [Accessed: 13 August 2023].

TABLE IV. LIST OF ALL FEATURES AT THE END OF DATA-PREPROCESSING STEP

Feature NO.	Name of Features	Description of Features
1	close	Price of Bitcoin at the end of one hour period. ¹
2	high	Highest price of Bitcoin in one hour period. ¹
3	low	Lowest price of Bitcoin in one hour period. ¹
4	open	Price of Bitcoin at the beginning of one hour period. ¹
5	forex_sentiment_score	Total sum of forex related news sentiment scores each day. ²
6	forex_total	Total number of forex news in each day. ²
7	stock_sentiment_score	Total sum of stock market-related news sentiment scores each day. ³
8	stock_total	Total number of stock market news in each day. ³
9	btc_sentiment_score	Total sum of Bitcoin related news sentiment scores in each day. ⁴
10	btc_Total	Total number of Bitcoin news in each day. ⁴

Feature NO.	Name of Features	Description of Features
11	eth_sentiment_score	Total sum of Ethereum related news sentiment scores in each day. ⁴
12	eth_Total	Total number of Ethereum news in each day. ⁴
13	Hash Rate	The average estimated number of hashes per second produced by the miners in the Bitcoin network. ⁵
14	Active Addresses	The number of unique Bitcoin addresses that were active in the network either as a sender or receiver. Only addresses that were active in successful transactions are counted. ⁵
15	Transaction Size (Total)	The total size of all Bitcoin transactions within the time period (in bytes). ⁵
16	Exchange Withdrawals	The total count of Bitcoin transfers from exchange addresses, i.e. the number of on-chain withdrawals from exchanges. ⁵
17	Difficulty	The current estimated number of hashes required to mine a block. ⁵
18	Transfer Volume (USDT)	The total amount of USDT transferred on-chain. Only successful transfers are counted. ⁵
19	Market Cap	The market capitalisation of Bitcoin as the product of the current supply by the current USD price. ⁵
20	Exchange Inflow Volume (Total) (USDT)	The total amount of USDT transferred to exchange addresses. ⁵
21	Exchange Outflow Volume (Total) (USDT)	The total amount of USDT transferred from exchange addresses. ⁵
22	Circulating Supply	The total amount of all USDT coins ever created/issued, i.e. the circulating supply. ⁵
23	Price (ETH)	Ethereum's closing price in USD. ⁵
24	Price (LTC)	Litecoin's closing price in USD. ⁵
25	Realized Cap HODL Waves 7y_10y	7 to 10 years HODL waves weighted by realized price. ⁵
26	Stock-to-Flow Ratio price	Stock to Flow is defined as the ratio of the current stock of a commodity (i.e. circulating Bitcoin supply) and the flow of new production (i.e. newly mined bitcoins). Bitcoin's price has historically followed the S/F Ratio and therefore it is a model that can be used to predict future Bitcoin valuations. (USD) ⁵
27	Stock-to-Flow Ratio ratio	Stock to Flow is defined as the ratio of the current stock of a commodity (i.e. circulating Bitcoin supply) and the flow of new production (i.e. newly mined bitcoins). Bitcoin's price has historically followed the S/F Ratio and therefore it is a model that can be used to predict future Bitcoin valuations. (Ratio) ⁵
28	000001.SS	SSE Composite Index ⁶
29	^AXJO	S&P/ASX 200 ⁶
30	^DJI	Dow Jones Industrial Average ⁶
31	^FTSE	FTSE 100 ⁶
32	^GDAXI	DAX PERFORMANCE-INDEX ⁶
33	^GSPC	S&P 500 ⁶
34	^IXIC	NASDAQ Composite
35	^N225	Nikkei 225 ⁶
36	^VIX	CBOE Volatility Index ⁶
37	CL=F	Crude Oil Sep 23 ⁶
38	GC=F	Gold Dec 23 ⁶
39	NG=F	Natural Gas Sep 23 ⁶
40	SI=F	Silver Sep 23 ⁶
41	ZW=F	Chicago SRW Wheat Futures, Dec-2 ⁶
42	DFF	[U.S] Federal Funds Effective Rate ⁷
43	CPIAUCSL	Consumer Price Index for All Urban Consumers: All Items in the U.S. City Average ⁷
44	SMA_24_hourly	Simple Moving Average (SMA) calculated over 24 hourly periods (daily) of Bitcoin close price in USD.
45	RSI_24_hourly	Relative Strength Index (RSI) calculated over 24 hourly periods (daily) of Bitcoin close price in USD.
46	MACD_hourly	Moving Average Convergence Divergence (MACD). MACD is calculated by subtracting the 26-period Exponential Moving Average (EMA) from the 12-period EMA. (Over Bitcoin Close Price in USD)
47	SMA_168_hourly	Simple Moving Average (SMA) calculated over 168 hourly periods (weekly) of Bitcoin close price in USD.
48	RSI_168_hourly	Relative Strength Index (RSI) calculated over 24 hourly periods (weekly) of Bitcoin close price in USD.

¹Bitstamp (2023). Bitstamp Public API (v2). [online] Available at: https://www.bitstamp.net/api/#ohlcv_data [Accessed 13 Aug. 2023].

²Forex News API (2023). Get the latest forex news from the best news sources. Use our API and access clean and relevant forex news data. [online]. Available at: <https://forexnewsapi.com> [Accessed 13 Aug. 2023].

³Stock News API (2023). Get the latest stock news from the best news sources. Use our API and access clean and relevant stock market news data. [online]. Available at: <https://stocknewsapi.com> [Accessed 13 Aug. 2023].

⁴Crypto News API (2023). Get the latest crypto news from the best news sources. Use our API and access clean and relevant crypto news data. [online]. Available at: <https://cryptonews-api.com> [Accessed 13 Aug. 2023].

⁵Glassnode Studio (2023). Metric Catalog. [online] Available at: <https://studio.glassnode.com/catalog> [Accessed: 13 August 2023].

⁶Yahoo Finance (2023). Trending Tickers. [online] Available at: <https://finance.yahoo.com/lookup/>.

⁷Federal Reserve Economic Data (2023). Welcome to FRED, your trusted source for economic data since 1991. [online] ECONOMIC RESEARCH FEDERAL RESERVE BANK of ST. LOUIS. Available at: <https://fred.stlouisfed.org> [Accessed 13 Aug. 2023].

Appendix B



Fig 1. Prediction of encoder only vanilla Transformer (with embedding) trained on top 5 features and Bitcoin close price



Fig 2. Prediction of encoder only vanilla Transformer (without embedding) trained on top 5 features and Bitcoin close price

Appendix C

TABLE V. HYPERPARAMETER TUNING RESULTS FOR VANILLA TRANSFORMER MODELS

Model Type	d_model_by_num_heads	d_ff	dropout_rate	lr	mlp_dropout_rate	mlp_size	num_heads	num_layers
Encoder Only Transformer	56	850	0.14	0.0001	0.15	62	1	7
Encoder-Decoder Transformer	44	50	0.13	0.0004	0.17	48	1	3

TABLE VI. HYPERPARAMETER TUNING RESULTS FOR PREFORMER MODELS

d_ff	d_layers	d_model	dropout	e_layers	learning_rate	n_heads
1024	1	512	0.28	2	0.001	8

Appendix D

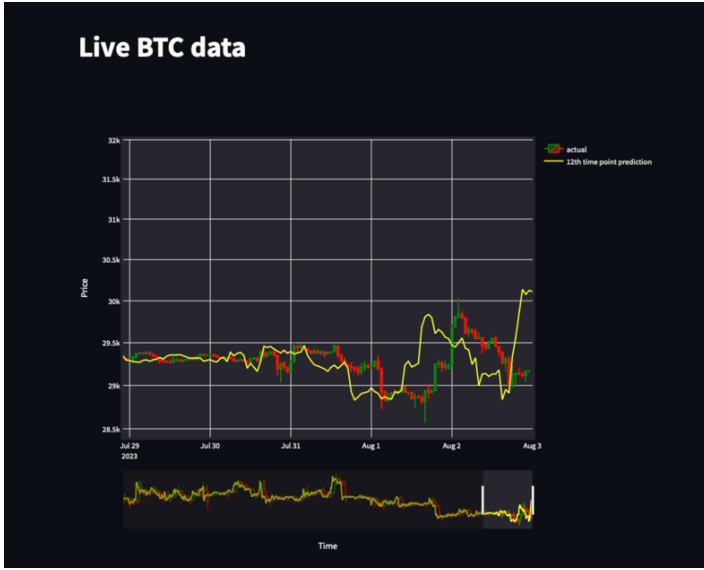


Fig 3. 12/12 I/O NLinear model prediction (trained only on BTC close price) on 3rd Aug. 2023



Fig 4. Realisation of 3rd Aug. prediction of 12/12 I/O NLinear model (trained only on BTC close price) on 9th Aug. 2023